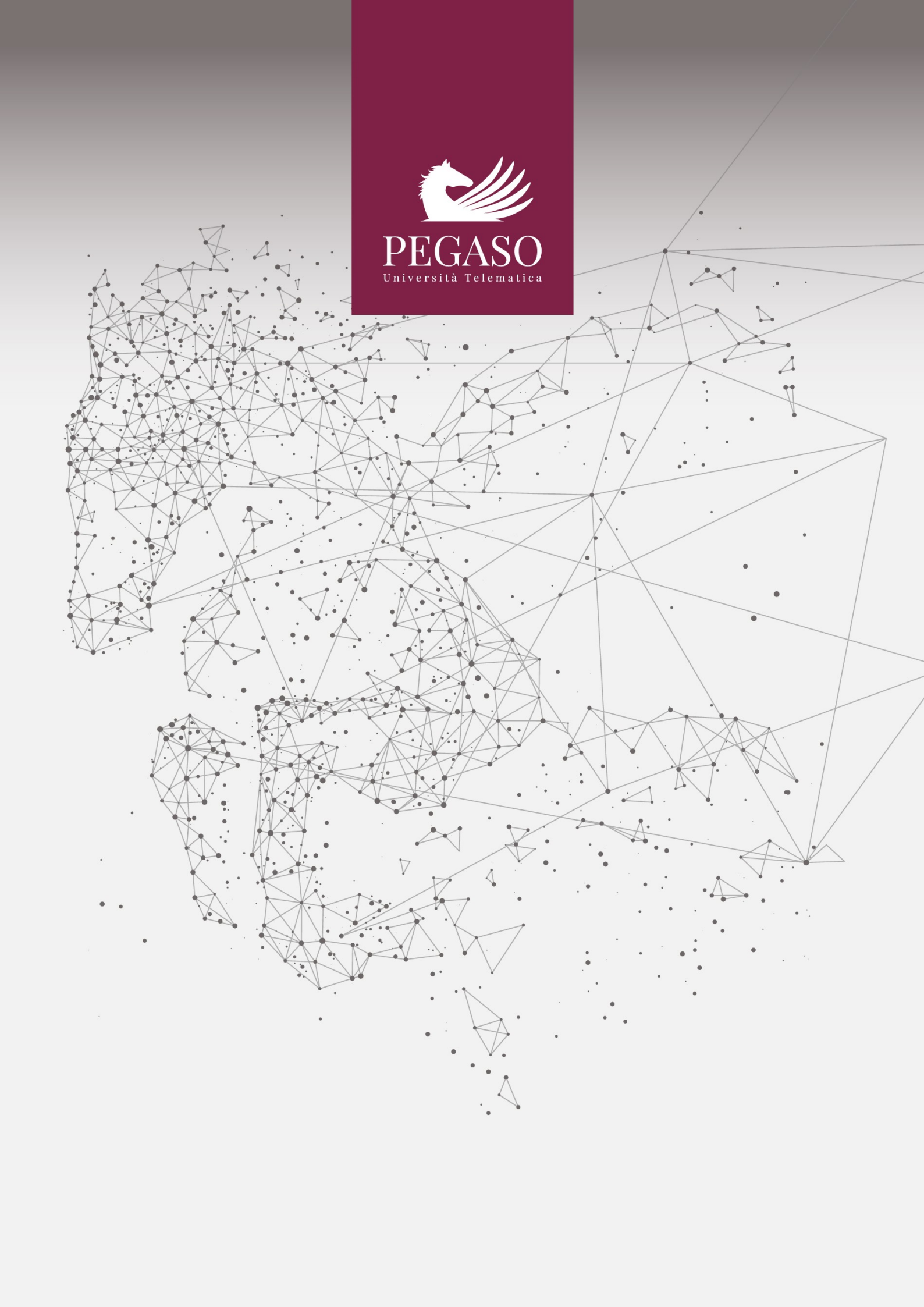

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. HTTP....3

2. HTTP REQUEST – HTTP RESPONSE....4

3. HTTP VERSIONS....9

BIBLIOGRAFIA E SITOGRAFIA....11

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. HTTP

L'**HyperText Transfer Protocol** (**HTTP**) è un protocollo a livello applicativo usato come principale sistema per la trasmissione d'informazioni sul web ovvero in un'architettura tipica **Client-Server**.

Le specifiche del protocollo sono gestite dal **World Wide Web Consortium** (**W3C**). Un server HTTP generalmente resta in ascolto delle richieste dei client sulla porta 80 usando il protocollo TCP a livello di trasporto.

La prima versione dell'HTTP, insieme con il linguaggio HTML e gli URL sono il nucleo fondante del World Wide Web (WWW), ed è stata sviluppata da Tim Berners-Lee al CERN di Ginevra: stiamo parlando a tutti gli effetti della nascita del Web.

Con la diffusione di NCSA Mosaic, un browser grafico di facile uso, il WWW conobbe un successo crescente e divennero evidenti alcuni limiti della versione 1.0 del protocollo, in particolare:

- l'impossibilità di ospitare più siti web sullo stesso server (virtual host);
- il mancato riuso delle connessioni disponibili;
- l'insufficienza dei meccanismi di sicurezza.

Ma cosa stanno a significare esattamente i singoli termini che compongono HTTP?

- **HyperText**: nato per l'interscambio di ipertesti, non è però limitato a questa specifica tipologia di contenuti.
- **Transfer**: pensato per il trasferimento (client-server) dei contenuti.
- **Protocol**: definisce le regole sintattiche, semantiche e di sincronizzazione con cui scambiare informazioni.

Come abbiamo detto in precedenza, l'HTTP è un protocollo che lavora in modo sincrono, con un'architettura di tipo **Client-Server**:

- il **client** invia una richiesta al server, specificando l'URL della risorsa che desidera recuperare;
- il **server** risponde con un messaggio di stato che indica se la richiesta è stata eseguita con successo o meno, e invia i dati richiesti al client.

Nell'uso comune il client corrisponde al browser ed il server la macchina su cui risiede il sito web. Vi sono quindi due tipi di messaggi HTTP:

- messaggi richiesta (detti HTTP requests);
- messaggi risposta (detti HTTP responses).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. HTTP Request – HTTP Response

Come introdotto nella sezione precedente, vi sono quindi due tipi di messaggi HTTP: messaggi richiesta (HTTP requests) e messaggi risposta (HTTP responses).

Un **messaggio** è composto di 3 parti:

- **Start line**: una stringa di testo, terminata da CRLF, che descrive il messaggio.
- **Header**: molteplici stringhe di testo (ciascuna terminata da CRLF) che definiscono opzioni e proprietà del messaggio; l'header termina con una riga vuota (contenente solo CRLF).
- **Body**: dati generici (anche binari) facenti parte dell'oggetto del messaggio.

I messaggi di tipo request, inviati dal client al server per richiedere un'azione, sono composti in questo modo:

- **Start line:** <method> <request-URL> <version>
- **Header:** <name1>: <value1> <name2>: <value2> ...
- **Body:** Dati binari o un semplice CRLF se non sono presenti dati
- Il <method> specificato nella start line può contenere i seguenti valori:
- **GET**: Chiede al server di inviare una certa risorsa al client
- **HEAD**: Simile a GET ma chiede al server di inviare soltanto l'header della risorsa e non il contenuto. Alcuni esempi di utilizzo sono:
    - Conoscere il tipo di una risorsa.
    - Controllare se una risorsa esiste.
    - Controllare se una risorsa è stata modificata dall'ultimo accesso.
- **PUT**: Inverso di GET. Chiede al server di memorizzare una certa risorsa inviata dal client.
- **POST**: Utilizzato per inviare dati generici al server. A differenza di PUT i dati non vanno memorizzati ma utilizzati da software esterni (esempio tipico inviare dati di form HTML).
- **TRACE**: Utilizzato per avere informazioni diagnostiche sulla topologia delle connessioni tra client e server (presenza di gateway, proxy, etc).
- **OPTIONS**: Chiede al server una lista delle funzionalità supportate. Utilizzata talvolta dal client per determinare il modo migliore per accedere alle risorse.
- **DELETE**: Utilizzata dal client per richiedere al server la rimozione di una certa risorsa. Come PUT è quasi sempre necessaria un'autenticazione.
- […]

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

I messaggi di tipo response sono inviati dal server al client in risposta ad un'azione richiesta nel precedente messaggio di request:

- **Start line**: <version> <status> <reason-phrase>
- **Header**: <name1>: <value1><name2>: <value2> ...
- **Body**: Dati binari o un semplice CRLF se non sono presenti dati

Nella Start Line, il campo <status> è un numero di 3 cifre che descrive il risultato della richiesta. La prima cifra descrive il tipo generico di stato, le ultime due lo stato specifico:

- Informational responses (100 – 199).
- Successful responses (200 – 299).
- Redirection messages (300 – 399).
- Client error responses (400 – 499).
- Server error responses (500 – 599).

I "classici" response code in cui ci si imbatte durante la navigazione sono:

- **200 OK**: Risposta standard per le richieste HTTP andate a buon fine.
- **301 Moved Permanently**: Questa e tutte le future richieste andranno dirette ad un altro URI (specificato nell'header Location).
- **307: Temporary Redirect**: In quest'occasione, la richiesta dovrebbe essere ripetuta con un'altra URI, ma successive richieste possono essere ancora dirette a quella originale. In contrasto con 303, la richiesta di POST originale deve essere reiterata con un'altra richiesta di tipo POST.
- **308 Permanent Redirect**: Questa richiesta e le future dovrebbero essere fatte verso un altro URI.
- **400 Bad Request**: La richiesta non può essere soddisfatta a causa di errori di sintassi.
- **401 Unauthorized**: Simile a 403/Forbidden, ma pensato per essere usato quando l'autenticazione è possibile ma è fallita o non può essere fornita.
- **403 Forbidden**: La richiesta è legittima ma il server si rifiuta di soddisfarla. Contrariamente al codice 401 Unauthorized, l'autenticazione non ha effetto.
- **404 Not Found**: La risorsa richiesta non è stata trovata ma in futuro potrebbe essere disponibile.
- **405 Method Not Allowed**: La richiesta è stata eseguita usando un metodo non permesso; ad esempio, questo accade quando si usa il metodo GET per inviare dati da presentare con un metodo POST.
- **408 Request Timeout**: Il tempo per inviare la richiesta è scaduto e il server ha terminato la connessione.
- **500 Internal Server Error**: Messaggio di errore generico senza alcun dettaglio.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **502 Bad Gateway**: Il server si sta comportando come un gateway o un proxy e ha ricevuto una risposta invalida dal server di upstream.
- **503 Service Unavailable**: Il server non è al momento disponibile. Generalmente è una condizione temporanea.
- **504 Gateway Timeout**: Il server si sta comportando come un gateway o un proxy e non ha ricevuto una risposta tempestiva dal server di upstream.

Eseguiamo una semplice richiesta sul nostro browser ed analizziamo cosa accade in termini http request-response:

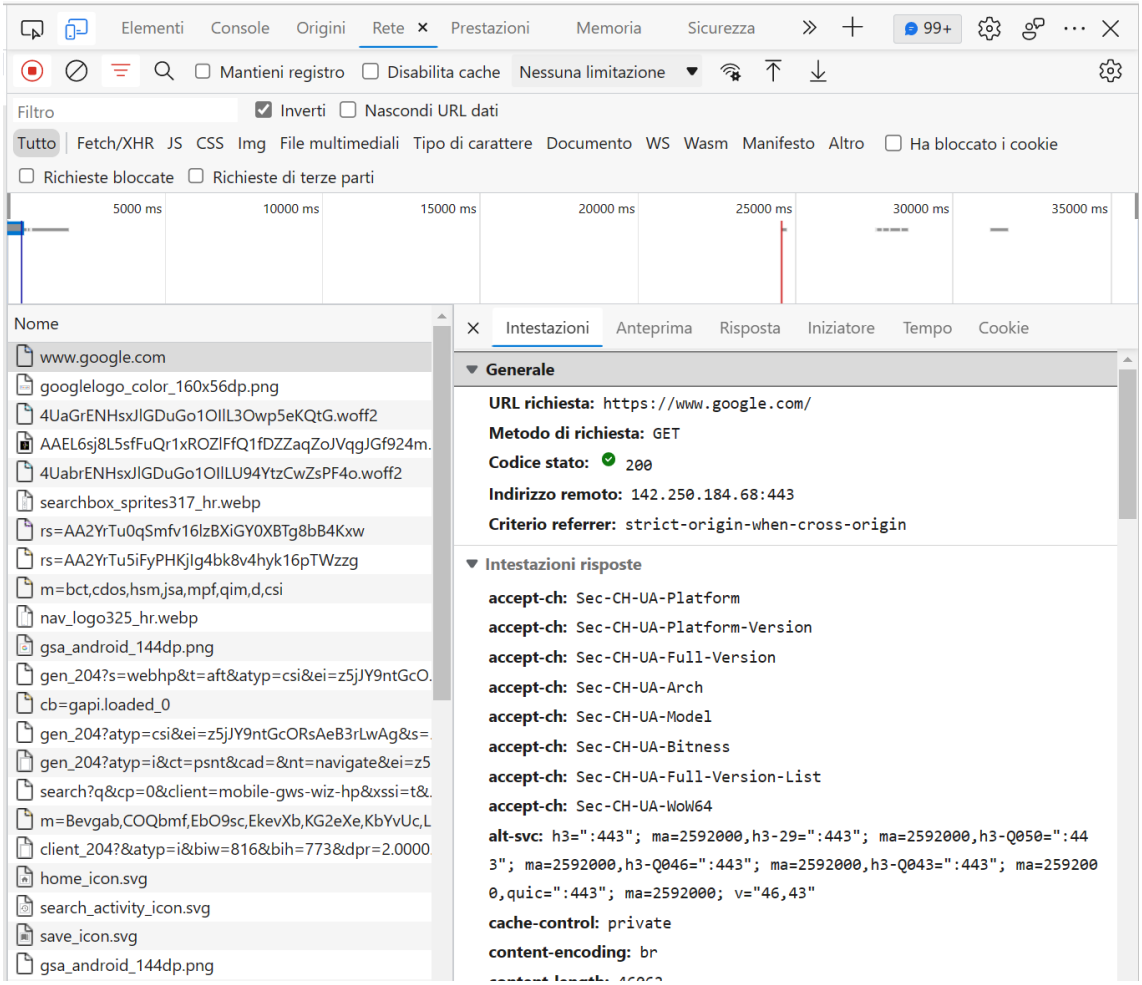


Figura 1 *- Microsoft Edge "esamina"*

In prima analisi è facile individuare che:

- È stato richiesto l'accesso alla pagina https://www.google.com
- Il metodo della richiesta è GET.
- Il codice della risposta è 200 OK.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- L'indirizzo IP corrispondente all'indirizzo www.google.com è 142.250.184.68
- La porta 443 indica che è stata fatta una richiesta https (protetta).

HTTP/1.1 definisce molteplici headers a seconda del tipo di messaggio e di contenuto da trasferire.

Si dividono in:

- Generali.
- Di richiesta (request).
- Di risposta (response).
- Di entità.
- Estensioni. Il protocollo tollera l'utilizzo di header al di fuori dello standard.
- Alcuni headers generali più importanti.
- **Date**: Data e ora di generazione del messaggio (non della risorsa).
- **Upgrade**: Specifica una nuova versione del protocollo che il mittente del messaggio desidera utilizzare.
- **Cache-Control**: Utilizzato per dare direttive sul caching.
- **Trailer**: Utilizzato per i trasferimenti "Chunked", cioè che coinvolgono più di un messaggio.

Nelle HTTP request, gli header più importanti sono:

- **Client-IP** Ip address della macchina client.
- **Host IP**: address della macchina server (verso cui si invia la richiesta).
- **Accept**: Utilizzato per informare il server sul tipo di files che il client supporta (ex. JPEG/PNG).
- **Accept-Encoding**: Utilizzato per informare il server sul tipo di codifica dei dati supportata (ex. Gzip).
- **If-Modified-Since**: Richiede al server di restituire la risorsa soltanto se è stata modificata successivamente ad una certa data.
- **Authorization**: Contiene i dati di autenticazione del client. Ad esempio, in basic access authentication le credenziali "username:password" sono codificate in BASE64 e inviate attraverso questo header.
- **Cookie**: Utilizzata dal client per inviare generici "token" al server, utili ad esempio per il meccanismo delle sessioni.

Nelle HTTP response, gli header più importanti sono:

- **WWW-Authenticate**: Lista di challenges dal server verso il client per invitarlo ad autenticarsi. Utilizzato nel basic access authentication.
- **Set-Cookie**: Utilizzata dal server per chiedere al client di memorizzare un generico token.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **Server**: Nome e la versione del software che agisce da web server.
- **Allow**: Lista dei metodi che possono essere richiesti su una certa risorsa.
- **Content-Encoding**: Encoding della risorsa (ex. gzip).
- **Content-Length**: Dimensione (numero di bytes) della risorsa inviata nel messaggio.
- **Content-Type**: MIME-type della risorsa.
- **Last-Modified**: Data di ultima modifica della risorsa.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. HTTP Versions

Nelle sezioni precedenti abbiamo parlato esplicitamente di "versioni" del protocollo HTTP; analizziamo nel dettaglio quali versioni del protocollo esistono:

- **HTTP/0.9** (1991): la prima versione del protocollo HTTP, che supportava solo il metodo GET e non prevedeva l'invio di intestazioni o la ricezione di codici di stato.
- **HTTP/1.0** (1996): questa versione introduceva l'utilizzo di intestazioni per la trasmissione di informazioni aggiuntive, come ad esempio l'autenticazione, e la ricezione di codici di stato per indicare l'esito della richiesta.
- **HTTP/1.1** (1997): questa versione introduceva l'utilizzo di connessioni persistenti, la possibilità di inviare richieste condizionali e ricevere risposte con codice 304 (Not Modified), la possibilità di codificare i dati trasmessi con gzip, l'autenticazione di base e digest e il metodo OPTIONS.
- **HTTP/2** (2015): questa versione introduceva il supporto per le richieste multiple in parallelo sulla stessa connessione, la compressione degli headers, la priorità delle richieste e la push delle risorse dal server al client.
- **HTTP/3** (2019): questa versione introduceva l'utilizzo di QUIC (Quick UDP Internet Connections) come trasporto per HTTP. QUIC è un protocollo di trasporto sviluppato da Google che si basa su UDP (User Datagram Protocol) anziché TCP (Transmission Control Protocol) per migliorare le prestazioni e la sicurezza delle connessioni.

Attualmente si sta lavorando ad una nuova versione di HTTP, chiamata **HTTP/3.1**. C'è da considerare che attualmente, il web adotta principalmente i protocolli HTTP/1.1 e HTTP/2. La maggior parte dei siti web noti usa HTTP/2 dato che è molto semplice da implementare. HTTP/2 è utilizzato da circa il 45% dei siti web e supportato da tutti i principali browser. Al contrario, HTTP/3 è attualmente utilizzato solo da circa il 5% dei siti web e non è ancora ben supportato da tutti i browser.

Rispetto ad HTTP/1.1, HTTP/2 fornisce in media un miglioramento delle prestazioni dal 5% al 15% sui tempi di caricamento della pagina. Inoltre, i miglioramenti delle prestazioni di HTTP/2 sono influenzati da diversi fattori che possono essere modificati da sviluppatori e webmaster. Ad esempio, pratiche come lo sharding del dominio, il numero di oggetti scaricati e domini di terze parti. Altri fattori non possono essere regolati facilmente perché cambiano con la connessione client – come la perdita di pacchetti o la latenza di rete – e sono al di fuori del nostro controllo.

Alcuni test mostrano che la consegna di oggetti di grandi dimensioni su HTTP/2 può essere più lenta di quella su HTTP/1.1 in caso di perdita di pacchetti. Questo perché HTTP/2 utilizza una singola connessione

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

TCP, rispetto a circa sei connessioni aperte dalla maggior parte dei browser su HTTP/1.1. Inoltre, gli algoritmi di controllo della congestione TCP hanno alcuni limiti e non sono sempre in grado di gestire ottimamente i carichi di traffico.

La soluzione a questo problema è utilizzare il protocollo HTTP/2 su un protocollo di trasporto diverso da TCP che fornisce un controllo della congestione più efficiente. Questo è, in sostanza, ciò che è HTTP/3: HTTP/2 su Quick UDP Internet Connections (QUIC) basato su User Datagram Protocol- (UDP).

Di seguito uno schema che mostra le differenze principali tra le varie versioni del protocollo:

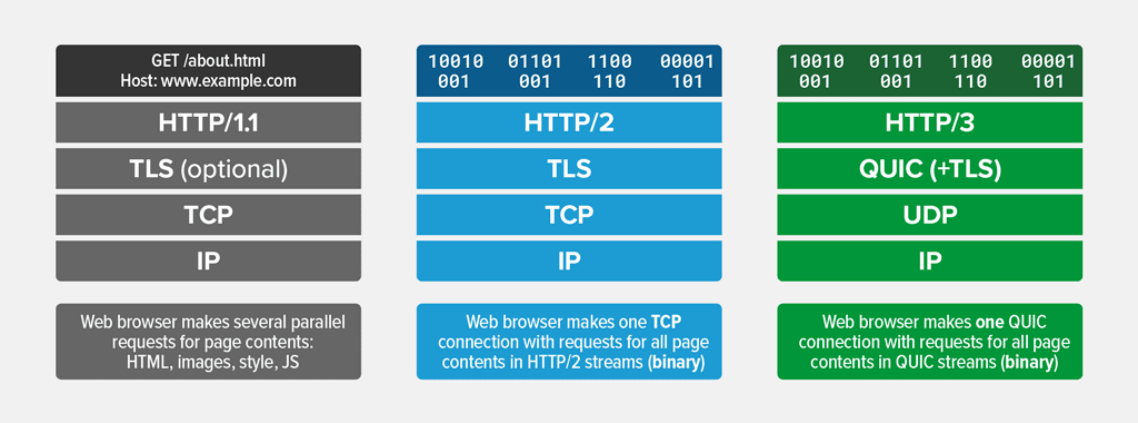


***Figura 2** - HTTP /1.1 - HTTP/2 - HTTP/3*

Da un punto di vista tecnico, una delle caratteristiche più significative che distingue HTTP/1.1 e HTTP/2 è il livello di inquadramento binario, che può essere considerato parte del livello dell'applicazione nello stack del protocollo Internet.

A differenza di HTTP/1.1, che mantiene tutte le richieste e le risposte in formato testo normale, HTTP/2 utilizza il livello di frame binario per incapsulare tutti i messaggi in formato binario, mantenendo comunque la semantica HTTP, come verbi, metodi e intestazioni.

Un'API a livello di applicazione creerebbe comunque messaggi nei formati HTTP convenzionali, ma il livello sottostante convertirà quindi questi messaggi in binario. Ciò garantisce che le applicazioni web create prima di HTTP/2 possano continuare a funzionare normalmente quando interagiscono con il nuovo protocollo.

La versione del protocollo HTTP in pratica definisce la grandezza e la velocità dell'autostrada che connette due luoghi: il web server ed il client. Più è larga l'autostrada e più mezzi potranno transitare contemporaneamente. maggiore è la velocità massima consentita e più rapidamente i mezzi arrivano a destinazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia e sitografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- F. Bergamasco - WEB APPLICATIONS AND TECHNOLOGIES - CT0142.
- https://www.evemilano.com/protocolli-http/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).